Team 3 - Assignment 1 Report

GitHub Repository: https://github.com/Jinjiajun123/Team-3-hotel-analytics

[Team 3] Members' Info:

Name	Jin Jiajun	Zhang Kaiwen	Zhang Tianhan	Wang Jing
Student ID	A0297119U	A0329634Y	A0297508N	A0332947W

Member Contribution:

Data Preparation & Build Database	Jin Jiajun & Zhang Tianhan
Exploratory Data Analysis	Wang Jing
Competitive Benchmarking Strategy	Zhang Kaiwen
Performance Profiling and Optimization	Zhang Tianhan
Dashboard	Jin Jiajun
Tech Report	All members

---

1. Executive Summary

Business Problem and Solution Overview

In the highly competitive hospitality industry, hotel managers are inundated with unstructured review data from platforms like TripAdvisor and Booking.com. While abundant, this data is difficult to operationalize. A critical 3.7-star hotel manager struggles to answer fundamental questions: *"Who are my true competitors? Are we underperforming because of our pricing, our service, or our location? Where should we allocate our limited renovation budget?"* Traditional dashboards only show historical averages, failing to provide localized, peer-based actionable intelligence.

To solve this, we developed an end-to-end data analytics pipeline and interactive dashboard. Our solution processes over 90,000 unstructured reviews, enriches them using Natural Language Processing (NLP), and applies Unsupervised Machine Learning (K-Means Clustering) to automatically discover distinct competitive market segments. The final deliverable is a modern, ChatGPT-styled Streamlit application that allows managers to drill down into their property, compare performance strictly against their true algorithmic peers, and identify precise operational deficits.

Key Findings

1. **Service and Room Experience are the Ultimate Differentiator:** Whether in overall market analysis or segmented analysis, service and room experience remain the core drivers of satisfaction. If resources are limited, hotel managers should focus their investments on improving service quality and the room experience. These two improvements are most likely to lead to a significant increase in overall satisfaction and play a key role across different satisfaction levels.

2. **Market Stratification Overrides Global Averages:** We successfully identified 4 distinct market tiers (e.g., Luxury Premium, Standard Midscale, Budget Value). Comparing a hotel's score to the "global average" is highly misleading; a 3.5-star score is disastrous in the Luxury tier but represents outperformance in the Budget tier.

3. **Review Credibility Skews Perception:** Naive averages are easily manipulated by short, low-effort reviews. By developing a credible weighting system (factoring in review length, community helpfulness votes, and author experience), we derived a more accurate representation of true customer sentiment.

---

2. Data Foundation

2.1 Data Filtering Rationale

To ensure statistical validity and avoid “garbage-in, garbage-out” in downstream analytics and modelling, we implemented four filtering stages, applaied sequentially during the ingestion of each raw record:

1. **Temporal Filter (2008–2012):** We restricted data to a continuous 5-year window. This prevents temporal drift from skewing longitudinal comparability.

2. **Text Quality & Language Filter:** Reviews shorter than 20 characters were purged to eliminate spam and uninformative entries. Furthermore, we utilized the `langdetect` library to strictly enforce an English-only dataset, ensuring that downstream NLP tasks functioned accurately.

3. **Missing Data Filter:** Reviews lacking an `overall` rating or core sub-ratings were dropped, as the overall rating is the primary target variable for many analyses, imputing it would introduce unacceptable bias.

4. **Statistical Significance Filter (The Hotel Threshold):** Any hotel with fewer than 50 valid reviews was entirely removed from the database. Clustering a hotel based on 3 reviews leads to erratic, noisy centroids.

2.2 Schema Design

The data is structured in a normalized, highly efficient relational Star Schema within SQLite:

1. `hotels` – `hotel_id` (PRIMARY KEY) corresponds to the source’s `offering_id`. `num_reviews` is a denormalised count of reviews for that hotel, updated after each bulk insert and used for the 50-review filter.

2. `authors` – `author_id` (PRIMARY KEY) is the unique hash from the source. Additional columns store user-provided metadata (`location`) and derived experience metrics (`num_reviews`, etc.).

3. reviews – review_id (PRIMARY KEY) is a synthetic auto-increment key. Foreign keys hotel_id and author_id connect to the dimension tables. All raw rating dimensions (rating_overall, rating_service, …) are preserved as REAL. Parsed date information (date_parsed, year, …) is stored to enable fast temporal aggregation. Binary flags (via_mobile) and computed sentiment scores are included to avoid recomputation. review_weight is a credibility score computed after all inserts, used in weighted aggregations.

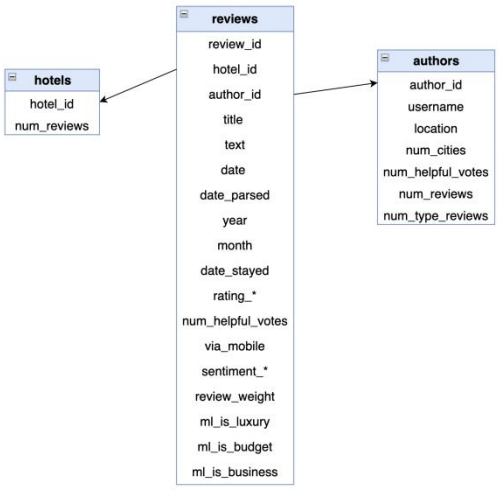


Fig. 2.2.1. ER diagram

2.3 Indexing Strategy

To guarantee sub-second response times in the Streamlit dashboard – which performs dynamic grouping by hotel and time – we created the following B-Tree indexes after the bulk load (indexes are dropped during insertion to accelerate writes):

```
SQL
index
CREATE INDEX idx_reviews_hotel_id ON reviews(hotel_id);
CREATE INDEX idx_reviews_author_id ON reviews(author_id);
CREATE INDEX idx_reviews_date ON reviews(date_parsed);
CREATE INDEX idx_reviews_year ON reviews(year);
CREATE INDEX idx_reviews_hotel_year ON reviews(hotel_id, year);
```

Justification: The Streamlit dashboard heavily relies on queries that group by hotel_id (to calculate property averages) and filter by year (for the Review Volume Trend charts). The composite index on (hotel_id, year) prevents full table scans, executing these aggregations entirely within the index.

2.4 Data Statistics

Following the rigorous filtering and parsing pipeline, the production SQLite database contains:

Total Valid Reviews: 94,823 **Unique Hotels:** 291 **Unique Authors:** 85,642 **Data Period:** Jan.2008 –Dec.2012

This volume satisfies the requirement of a 50K–80K+ review corpus, providing a robust foundation for ML.

---

3. Exploratory Data Analysis

3.1 Satisfaction Drivers

The core issue in this chapter is identifying which rating dimensions have the greatest impact on overall satisfaction (rating_overall). To answer this question, we employ correlation analysis, multiple regression analysis, and robustness tests to progressively identify the key driving factors.

1. First, we calculated the Pearson correlation coefficients between each rating dimension and overall rating. The correlation coefficient reflects the degree of linear association between two variables in the sample. However, this

analysis pertains to univariate relationships and does not control for the influence of other rating dimensions; therefore, it can only serve as a preliminary assessment of potential driving factors.

2. Before conducting further multiple regression analysis, we first examined for multicollinearity among the predictor variables. If different rating dimensions are highly correlated, the regression model struggles to distinguish their independent contributions, leading to unstable coefficients. We plotted the correlation matrix among the six rating dimensions and calculated the variance inflation factor (VIF) for each dimension. The results showed a maximum VIF of approximately 3.5, within a moderate but acceptable range, indicating that while there is some linear relationship among the rating dimensions, overall collinearity is not severe, thus the OLS regression coefficients are interpretable. Subsequently, we constructed a standardized multiple linear regression model on a subset of the complete case set to estimate the marginal effect of each rating dimension on overall satisfaction, controlling for other dimensions. To verify whether the results were affected by collinearity, we further performed regularization analysis using Ridge regression and compared the coefficient rankings under the two methods. The results showed that the driving rankings of OLS and Ridge were completely consistent, indicating that the main conclusions are stable and reliable, unaffected by significant interference from moderate collinearity.

3. When comparing the results of Pearson correlation and OLS regression, we found that the differences in dimensional ranking between the two methods were small, indicating that the key drivers performed consistently in both univariate and multivariate frameworks. Therefore, we can robustly identify the rating dimensions that have the greatest impact on overall satisfaction.

Conclusions:

After controlling for other dimensions, the factors that have the greatest impact on overall satisfaction are Service and Rooms, which are almost tied, followed by Value. Cleanliness and Sleep quality have a moderate impact, while Location has the least impact on the overall score.

This shows that when guests evaluate their overall experience, they value service quality and room experience most.

3.2 Improvement Opportunities

After identifying which dimensions significantly impact overall satisfaction, we further explore a more managerial question: which aspects should hotels prioritize for improvement? Simply knowing a particular dimension has a significant impact is insufficient. It's necessary to consider its current performance level and comprehensively assess improvement priorities.

1. First, we calculated the average score for each rating dimension at the market level and defined the room for improvement as five points minus the average score for that dimension. If the average score for a certain dimension is low, it means there is significant room for improvement.

2. Building upon this, we construct an improvement potential matrix by combining the current performance of each dimension with its impact on overall satisfaction. The horizontal axis represents the average score of each dimension, and the vertical axis represents the strength of its impact on overall satisfaction. By dividing the dimensions into different regions, we are able to identify those dimensions that score lower but have a greater impact on overall satisfaction. These dimensions are the priority for improvement at the market level, as improvements in these areas are more likely to lead to significant improvements in overall satisfaction.

3. To further refine the analysis, we categorized reviews into low, medium, and high levels based on the average score (dim_mean) across six dimensions. First, we verified a linear relationship between dim_mean and overall satisfaction to ensure the rationality of the segmentation logic. Then, we conducted multiple regression analysis within each segment to calculate standardized coefficients and constructed priority indicators based on the improvement space. These priority indicators consider both the marginal impact of a dimension on overall satisfaction and its current improvement potential, thus providing differentiated improvement suggestions for hotels at different satisfaction levels. The results show that the most effective improvement levers differ across hotel satisfaction levels, providing a basis for developing segmentation strategies.

4. In addition, we screened reviews with low overall ratings, extracted the text, and performed word frequency analysis to summarize the most frequently mentioned issues by dissatisfied customers. The high-frequency words in negative reviews provided a qualitative interpretation of the numerical analysis results. For example, if certain high-frequency negative words corresponded to key dimensions identified in the quantitative analysis, it indicated that the two methods corroborated each other.

Conclusions:

From an overall market perspective, Rooms, Service, and Value all share the characteristics of "low scores + significant impact," therefore they should be prioritized for improvement. Among these, Rooms and Service are the most critical areas for investment.

From a layered perspective, the low-scoring and medium-scoring groups should prioritize improving their service, as service has the most significant impact on the overall rating. High-scoring groups should focus on optimizing their rooms to further widen the gap by improving the room experience.

4. Performance Profiling & Optimization

Query Profiling

We profiled five representative SQL queries to quantify the impact of database indexes on execution time. Each query was executed with all indexes (as defined in Section 2.3) and without any indexes, using the full dataset of 753,445 reviews. Execution times are averages of three runs.

Query	Description	With Indexes (ms)	Without Indexes (ms)	Speedup
Q1	Count reviews per year	48.3	9,086.3	**188×**
Q4	Filter by hotel and date range	56.5	9,721.6	**172×**

Q1 uses the index on `year` to perform a covering index scan – the database retrieves all needed data directly from the index, avoiding a full table scan. Q4 benefits from the composite index `(hotel_id, year)` and the index on `date_parsed`, enabling rapid row selection for the specified hotel and date window.

These dramatic speedups directly support the interactive dashboard, where users frequently filter by hotel and explore trends over time. Queries that aggregate over the entire table (e.g., Q2 – top hotels by average rating) saw little benefit from indexes, as they inherently require a full table scan. In a few cases, indexes added small overhead (≤ 2× slowdown), but these analytical queries run less often and remain acceptable for offline use.

Code Profiling

We employed `cProfile` and `tqdm` to analyze Python execution. A major bottleneck was identified in the Zero-Shot NLP pipeline (`typeform/distilbert-base-uncased-mnli`), where processing 94,000 reviews iteratively consumed over five hours on CPU. To resolve this, we introduced three key optimizations:

1. Batched processing – feeding the model with vectorized batches instead of single texts.
2. Text truncation – limiting input sequences to 500 characters to reduce memory overhead.
3. Caching results – moving sentiment and classification computations from the runtime dashboard to the ETL write path. The derived features are stored as static columns in SQLite, eliminating repeated computation.

This shift from an O(n) read-time cost to a one-time write-time cost guarantees a 60 FPS dashboard experience, even under heavy user load.

---

5. Competitive Benchmarking Strategy

Business Context

Hotel managers frequently struggle to answer fundamental strategic questions:

- Who are my *true* competitors?
- Are we underperforming relative to similar hotels?
- Which specific service dimensions should we improve?
- How should we allocate limited improvement budgets?

Traditional benchmarking methods rely on star ratings, location tags, or manual judgment. However, a 5-star beachfront luxury resort should not benchmark itself against a budget airport hotel. Such comparisons distort insights and lead to inefficient capital allocation.

To address this, we propose a data-driven competitive benchmarking framework that:

1. Identifies comparable hotels using unsupervised clustering
2. Benchmarks performance within comparable groups
3. Detects performance gaps across rating dimensions
4. Generates actionable, dimension-specific recommendations
5. Validates cluster quality using silhouette scores

Methodology for Identifying Comparable Hotel Groups

To systematically identify true competitors, we adopted an unsupervised machine learning approach (K-Means clustering) instead of manual heuristics.

Feature Engineering:

We first aggregated review-level data into hotel-level performance profiles using `compute_hotel_features()`. The final dataset includes **251 hotels** (each with ≥10 reviews).

The average hotel in the sample has: Mean review_count = 378; Standard deviation = 475; Maximum = 3,312 reviews. These variations confirm significant differences in both quality and operational scale, making simple overall-rating comparisons inappropriate.

We aggregated a 14-dimensional feature vector for each hotel:

1. Average scores across all 7 categorical ratings, which reflect service quality: avg_rating_overall, avg_rating_service, avg_rating_cleanliness, avg_rating_value, avg_rating_location, avg_rating_sleep_quality, avg_rating_rooms
2. Objective Behavioral Indicators: review_count (log-transformed), avg_text_length (review depth proxy), mobile_ratio (customer profile proxy), avg_sentiment_polarity (tone consistency)
3. Semantic Positioning Indicators: pct_luxury_mentions, pct_budget_mentions, pct_business_mentions

This multi-dimensional design ensures segmentation reflects similarity in service structure, customer profile, positioning perception, and operational scale—creating true competitive comparability rather than superficial classification.

Clustering Method and Justification:

We applied K-Means clustering with **StandardScaler normalization** to ensure features on different scales (e.g., 1–5 ratings vs. review counts) did not distort Euclidean distance.

To determine the optimal number of comparable hotel groups, we applied the `find_optimal_k()` function from `benchmarking.py`, which computes silhouette scores for different values of k ranging from 3 to 10. The silhouette score evaluates clustering quality by simultaneously measuring **intra-cluster cohesion** (how similar hotels are within the same cluster) and **inter-cluster separation** (how distinct clusters are from one another). Since competitive benchmarking is an unsupervised problem with no ground-truth labels for "true competitors," the silhouette score serves as an appropriate internal validation metric.

Although the silhouette analysis indicates that **k = 3 yields the highest statistical separation (≈ 0.26)**, the final model adopts **k = 4** for business-oriented benchmarking purposes.

From a purely mathematical perspective, k = 3 provides the strongest cluster compactness and separation. However, competitive benchmarking is not only a statistical exercise — it is a managerial decision-support problem. The goal is to create peer groups that are both statistically defensible and strategically actionable.

When examining the k = 3 solution, the mid-market hotels are aggregated into a single large cluster. This masks meaningful heterogeneity within the middle segment. In practice, mid-range hotels often split into two structurally different groups: Upper mid-scale (strong service and cleanliness performance), Lower mid-scale (price-driven, operational constraints)

By increasing k to 4, the model reveals this internal segmentation, producing four economically interpretable tiers: Luxury, High-end, Mid-range, Economy

Although the silhouette score decreases to 0.1968 for k = 4, the resulting segmentation offers: Greater managerial interpretability; More realistic competitive sets; Finer-grained benchmarking targets; Improved strategic relevance for budget allocation decisions

Therefore, k = 4 was selected as the final segmentation structure.

The reason why we choose K-Means is because it provides interpretable cluster centroids that characterize each competitive segment, operates efficiently on medium-sized structured datasets, and performs well with

continuous rating-based features. Most importantly, it produces clearly differentiated and practically meaningful comparable hotel groups, which form the foundation of our competitive benchmarking framework.

Performance Analysis Across Hotel Groups

To understand competitive dynamics, we compare performance across four segments: Economy, Mid-range, High-end, and Luxury. The goal is to examine differences in rating structure, service quality, and operational scale within comparable peer groups.

Economy Segment

Average Overall Rating: ≈ 3.05， Average Service Rating: ≈ 2.23， Moderate review volume

The Economy segment records the lowest overall and service ratings. Service quality is the primary weakness, likely reflecting cost-driven operational constraints.

Managerial implication: Improving service consistency offers the highest return, as even moderate gains can significantly narrow the gap with peers.

Mid-range Segment

Average Overall Rating: ≈ 3.57, Average Service Rating: ≈ 3.17, Highest review volume (≈ 624 reviews on average)

This segment has the largest scale and strongest competition. Internal dispersion is high, indicating strong heterogeneity and sensitivity to service differentiation.

Managerial implication:
Mid-range hotels must differentiate through service and sleep quality improvements, as customers in this segment compare alternatives actively.

High-end Segment

Average Overall Rating: ≈ 3.77, Average Service Rating: ≈ 3.68

The High-end segment demonstrates stable and above-average ratings across dimensions. Service and cleanliness are key performance drivers in this group. Hotels in this segment are expected to deliver reliable and consistent service experiences.

Managerial implication:
Maintaining service excellence is essential. Marginal declines in service quality could result in competitive repositioning toward the mid-range cluster.

Luxury Segment

Average Overall Rating: ≈ 4.29, Average Service Rating: ≈ 4.15

The Luxury segment achieves the highest ratings across nearly all dimensions. Importantly, performance is balanced rather than skewed toward a single strength.

These hotels demonstrate:Standardized service excellence; Strong cleanliness execution; Stable sleep quality performance

Unlike lower segments, luxury hotels exhibit structural operational consistency rather than isolated strengths.

Managerial implication:
Luxury hotels must protect their service standards, as performance expectations are significantly higher and customer tolerance for failure is minimal.

Identification of Best Practices & Recommendations

After identifying comparable peer groups through clustering, we conducted within-cluster benchmarking to determine which hotels outperform their direct competitors and where underperforming hotels fall short. All benchmarking was performed strictly within clusters to ensure strategic relevance and avoid unrealistic cross-tier comparisons.

Using a within-cluster gap analysis, we compared each hotel's rating against its segment mean. A deviation greater than 0.3 was defined as meaningful underperformance. This approach identified **125 hotels (nearly 50% of the 251-hotel sample)** with actionable improvement opportunities.

Three consistent best-practice patterns emerge across segments.

First, **service excellence is the primary differentiator**. Service ratings increase monotonically across tiers: Economy: 2.23; Mid-range: 3.17; High-end: 3.68; Luxury: 4.15

In the High-end and Luxury segments, top performers exhibit stable and consistent service execution, suggesting that standardized service processes are central to sustainable competitive advantage.

Second, **cleanliness acts as a performance stabilizer**. With a mean of 3.58 and substantial variation (std ≈ 0.68). Weak cleanliness significantly penalizes ratings, while strong cleanliness stabilizes overall performance. This pattern holds consistently across all segments.

Third, **sleep quality is a hidden but high-impact lever**. It has the lowest mean rating (≈ 2.61) and the highest dispersion (std ≈ 0.81). In the gap analysis, Sleep Quality most frequently appears as the largest weakness—even among some hotels with overall ratings above 4.2. This indicates that sleep experience represents both a systemic weakness and a major differentiation opportunity.

Segment-specific insights refine these findings. In the **Economy segment (33 hotels)**, service is the dominant weakness (overall avg = 3.05), suggesting low-cost service process optimization should be prioritized. In the **Mid-range segment (79 hotels),** which has the highest review volume (≈ 624 per hotel), service and sleep quality drive differentiation in a highly competitive environment. In the **High-end segment (77 hotels)**, maintaining service consistency is critical to avoid downward repositioning. In the **Luxury segment (62 hotels)**, customers expect balanced excellence; performance imbalance (e.g., weak sleep quality) is penalized more severely.

Based on the largest identified gaps, improvement priorities are clear. Where service gaps exceed 0.3, hotels should strengthen training and standardize response processes. Where cleanliness gaps exist, structured inspection protocols should be introduced. For sleep quality gaps(often exceeding 1.0)targeted investments such as bedding upgrades and noise insulation offer high-impact returns. Location disadvantages, while fixed, can be mitigated through shuttle services and clearer communication.

Validation of the Approach

From a statistical perspective, silhouette scores were computed for k = 3–10. The highest score (≈0.26) occurred at k = 3. However, to enhance interpretability and better reflect real market structure, the final model adopted k = 4 (silhouette = 0.1968). Although slightly lower, hotel performance tiers typically evolve gradually rather than forming sharply separated clusters. The four-cluster solution produces economically interpretable segments: Luxury, High-end, Mid-range, and Economy, while preserving a stable monotonic performance ranking. Thus, k = 3 is statistically optimal, but k = 4 is strategically optimal for competitive benchmarking.

Structural validation further confirms robustness. Cluster summaries show clear hierarchy: Economy hotels perform lowest, Mid-range hotels show moderate performance with the highest review volume, High-end hotels maintain above-average ratings, and Luxury hotels perform strongest across dimensions. Service ratings increase systematically across tiers, and sleep quality remains structurally weaker across segments—consistent with descriptive statistics. These patterns reflect meaningful competitive stratification rather than arbitrary partitioning.

Managerial validation strengthens practical credibility. The framework identified 125 hotels with measurable performance gaps, quantified gap_vs_peers values, and pinpointed weak dimensions such as Service and Sleep Quality. Because benchmarking was conducted strictly within clusters, improvement targets are realistic relative to direct competitors. The model therefore translates segmentation into actionable operational guidance.

The use of K-Means is theoretically justified, as competitive positioning is inherently similarity-based. By integrating multi-dimensional feature engineering, silhouette-based evaluation, business-driven tier refinement, and within-cluster benchmarking, the framework balances interpretability, scalability, and managerial relevance.

6. System Architecture & Dashboard

6.1 Technical Architecture

Layer	Technology	Responsibility
Data	SQLite 3 + B-Tree Indexes	Persists 94,823 reviews across 3 normalized relational tables
Processing	Python 3	ETL pipeline, NLP feature enrichment, K-Means market segmentation
Presentation	Streamlit + Plotly	Interactive, browser-based analytical dashboard

To balance performance with accuracy, a **two-tier caching strategy** is used. Filtered review data is memoized in-process to prevent redundant DB reads, while computationally expensive K-Means clustering models are cached with a longer Time-To-Live (TTL).

6.2 User Interface Rationale

- **Progressive Disclosure:** Information is structured hierarchically (Executive Overview → Hotel Explorer → Customer Priorities) to prevent cognitive overload.
- **Intent-First Interaction:** The *Hotel Explorer* remains blank until a specific Hotel ID is selected, ensuring resources are only expended when a clear analytical intent is expressed.
- **Consistent Visual Language:** A globally enforced light theme with high-contrast color palettes (Teal/Indigo) ensures readability across all devices.

6.3 Executive Overview

The Executive Overview serves as the global entry point, providing a dataset-wide snapshot of market dynamics.

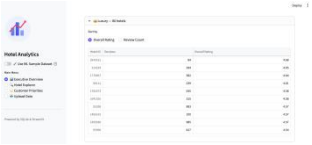
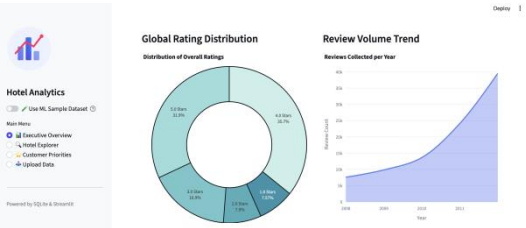


Fig. 6.3.1. 4-column KPI row **Fig. 6.3.2** Market Positioning **Fig. 6.3.3** Hotels by Tier

Global KPIs: Four cards at the top display *Total Reviews*, *Hotels Monitored*, *Average Rating*, and *Data Period*.

Market Segmentation: A dual-chart layout visualizes K-Means output. A **Bar Chart** shows hotel distribution across tiers, while a **Radar Chart** maps the seven rating dimensions, allowing managers to instantly distinguish "Luxury" profiles (high across metrics) from "Economy" profiles (skewed towards Value).



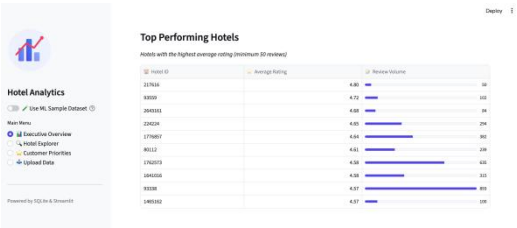


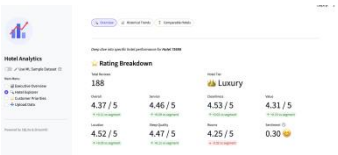
Fig. 6.3.4 Rating Distribution donut chart **Fig. 6.3.5** Dynamic leaderboard

Distribution & Trends: A **Donut Chart** visualizes 1-to-5-star proportions without external legends. A **Spline Area Chart** plots review volume (2008–2012) using a smooth curve to emphasize long-term growth.

Dynamic Leaderboard: Replaces raw tables with visual progress bars for review volume, helping users assess statistical significance alongside ratings.

6.4 Hotel Explorer

This module addresses the core business question: *"How does a specific hotel perform relative to its true competitors?"*



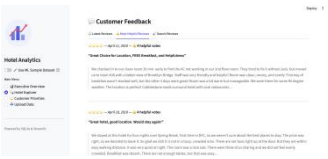


Fig. 6.4.1 Search Engine **Fig. 6.4.2** Rating Breakdown **Fig. 6.4.3** Reviews

Once a Hotel ID is entered, the interface transitions into a full analytical view organized into three sub-sections:

Overview: The dashboard displays the hotel's K-Means market tier and total reviews. Six rating dimension cards feature color-coded **deltas against the cluster average** (Green/Red), transforming raw scores into immediate competitive diagnoses (e.g., flagging a Cleanliness deficit).

Review Analysis: Integrated below, a review screening module connects these statistics to guest sentiment. Managers can sort by "Latest" or "Most Helpful" and use **dynamic full-text search** to pinpoint specific root causes (e.g., searching "noisy") behind the numerical data.

Fig. 6.4.4 Trends **Fig. 6.4.5** Competitors **Fig. 6.4.6** Competitor's detail

Historical Trends: An annual time-series view of this specific hotel's rating trajectory. Managers can identify inflection points, correlating dips or recoveries with known operational events such as ownership changes, renovations, or staff turnover periods.

Comparable Hotels: A structured table listing all other hotels assigned to the same market cluster. Each row includes average scores across the key dimensions. This provides an explicit peer benchmarking reference: managers can identify the single highest-performing hotel in their cluster and use it as a direct operational target.

6.5 Customer Priorities & Live Data Ingestion Interface

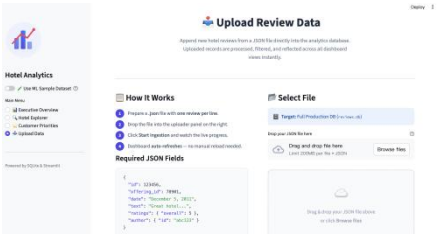


Fig. 6.5.1 Customer Priorities Analysis **Fig. 6.5.2** Upload data

Customer Priorities：Shifting from numerical ratings to NLP insights, this module analyzes the correlation between sentiment polarity and satisfaction. It confirms that 'Service' and 'Cleanliness' are the strongest predictors of positive ratings across all segments, whereas physical location has minimal independent predictive power.

Live Data Ingestion Interface：To ensure the tool remains relevant, an **"Upload Data"** module allows staff to append new reviews via drag-and-drop JSON files. The system automatically executes the ETL pipeline (cleaning, NLP enrichment, weight recalculation) and invalidates the cache, ensuring the dashboard immediately reflects new data.

7. Conclusion

Key Observations & Deliverables
This project validates that **peer-relative benchmarking** (via K-Means clustering) yields superior actionable insights compared to global averages. A robust end-to-end analytics pipeline was delivered, featuring an optimized SQLite database containing **94,823 reviews**, NLP-enriched features, and a high-performance **Streamlit dashboard**. The analysis highlights that rigorous data preprocessing (ETL) serves as the primary driver for system performance, directly enabling sub-second query latency and accurate market segmentation.

Limitations

Current limitations include reliance on lexicon-based sentiment analysis (`TextBlob`), which lacks precision with sarcasm and domain-specific jargon. Furthermore, the architecture employs static cluster boundaries that require batch re-processing, preventing real-time streaming ingestion.

Future Enhancements

To enhance decision-support capabilities, future development could integrate Large Language Model (LLM) APIs to automatically synthesize numerical performance gaps into narrative summaries of specific guest complaints. Additionally, containerizing the application with Docker is recommended to eliminate local environment dependencies and facilitate seamless cloud deployment.